

大数据综合项目实战

第 2 讲：把项目数据真正落到 HDFS

从“本地文件”到“分布式存储”的第一次完整闭环

数据从 Starter Pack 到 HDFS 的完整路径



每一步都要有“可观察证据”，不要只相信命令没有报错。

本讲最终目标

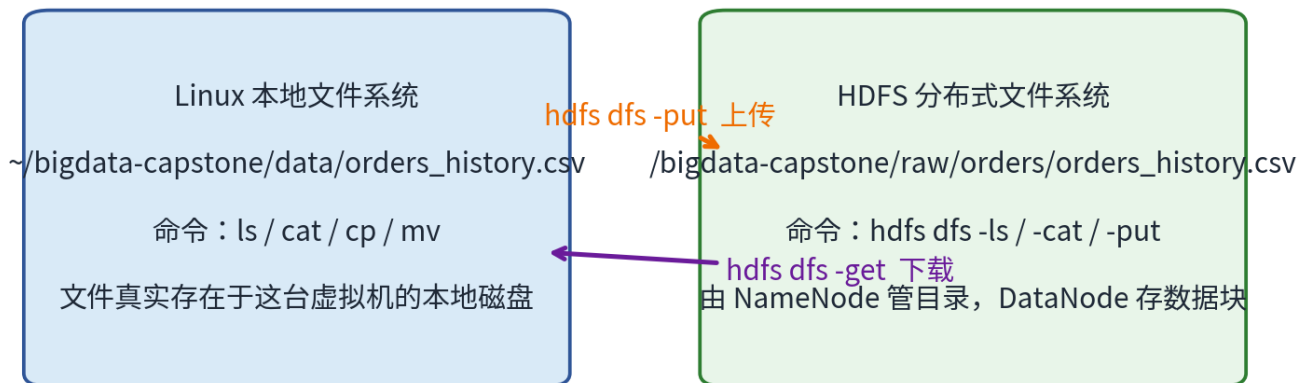
你不仅要把 users.csv、products.csv、orders_history.csv 上传到 HDFS，还要能证明：服务正常、路径正确、文件内容一致、Block/副本状态可解释，并知道第二天怎样重新启动继续做实验。

统一环境：Ubuntu 22.04 + Hadoop 3.3.5 + JDK 1.8；操作入口统一使用 Windows VS Code Remote SSH 连接 Ubuntu 虚拟机。

1. 先搞清楚：今天真正发生了什么

第 1 讲的数据还只是普通 Linux 文件。今天我们要把这些“本地文件”复制到 HDFS，让后面的 MapReduce 和 Spark 都从同一份分布式数据源读取。

本地 Linux 文件系统与 HDFS 是两套“命名空间”



关键：Linux 的 ls 看不到 HDFS 文件；HDFS 的 hdfs dfs -ls 也不是在看本地目录。

最容易犯的错

在 Linux 终端执行 ls 看到了文件，就以为 HDFS 里也有；或者执行 hdfs dfs -ls 后找不到 ~/bigdata-capstone。两套文件系统的路径不是一回事。

一个非常实用的比喻：本地文件系统像“你电脑桌面上的文件柜”；HDFS 像“学校图书馆的目录系统”。把文件放在桌面，并不等于图书馆已经收藏；必须通过 hdfs dfs -put 完成“入库”。

2. 本讲完成标准：做到这 8 件事才算完成

- 能在 VS Code 中确认自己连接的是 Ubuntu 虚拟机，而不是 Windows 本机。
- 能确认 java、hadoop、hdfs、yarn 命令可用。
- 能在服务初始未启动的情况下，正确启动 HDFS 和 YARN。
- 能用 jps、hdfs dfsadmin -report、yarn node -list 三类证据验证服务。
- 能设计并创建 /bigdata-capstone/raw/... HDFS 目录。
- 能上传三张 CSV 表，并用 -ls、-cat、-du、wc -l 验证。
- 能用 hdfs fsck 观察 Block 与存放位置，并解释“小文件不等于硬占 128 MB”。
- 能安全停止服务，并知道第二天重新打开虚拟机后从哪里继续。

3. 保姆式起步：从打开虚拟机和 VS Code 开始

3.1 启动 Ubuntu 虚拟机

先启动你的 Hadoop Ubuntu 虚拟机。不要急着开 Hadoop 服务；本课程默认“每次课重新打开虚拟机，服务初始未启动”。

判断虚拟机已就绪

你至少应该能看到 Ubuntu 已启动，并且网络正常。若你使用固定 IP，后面仍可继续使用 VS Code 中保存的 `hadoop@<IP> Remote SSH` 连接。

3.2 在 Windows 打开 VS Code 并连接 Remote SSH

- 打开 VS Code。
- 观察左下角 Remote 状态；若还未连接，点击左下角远程按钮。
- 选择 Connect to Host...，选择你之前配置的 `hadoop@<虚拟机 IP>`。
- 等待新窗口打开。左下角应出现类似 `SSH: <虚拟机 IP>`。
- 点击 File → Open Folder，打开 `/home/hadoop/bigdata-capstone`（若你的用户名不是 `hadoop`，请替换成自己的 home 路径）。
- 打开 Terminal → New Terminal。后面的命令全部在这个“远程 Ubuntu 终端”执行。

先确认“我是谁、我在哪台机器、我在哪个目录”

```
whoami
hostname
pwd
```

进入项目目录（如果当前不在此处）

```
cd ~/bigdata-capstone
```

看第 1 讲的数据是否还在

```
ls -lah
ls -lah data
```

你应该看到什么

`data` 目录中至少有 `users.csv`、`products.csv`、`orders_history.csv`、`order_events.jsonl`。若这里都没有数据，不要继续 HDFS；先回到第 1 讲把 Starter Pack 解压正确。

4. 启动前检查：先确认环境，再启动服务

很多初学者遇到错误后直接反复 `start-dfs.sh`。更稳妥的方法是：先确认命令和配置，再启动守护进程。

```
# 1) Java
java -version

# 2) Hadoop
hadoop version

# 3) 命令是否在 PATH 中
which hadoop
which hdfs
which yarn

# 4) 看 HDFS 默认地址
hdfs getconf -confKey fs.defaultFS

# 5) 看当前是否已有 Java 守护进程
jps
```

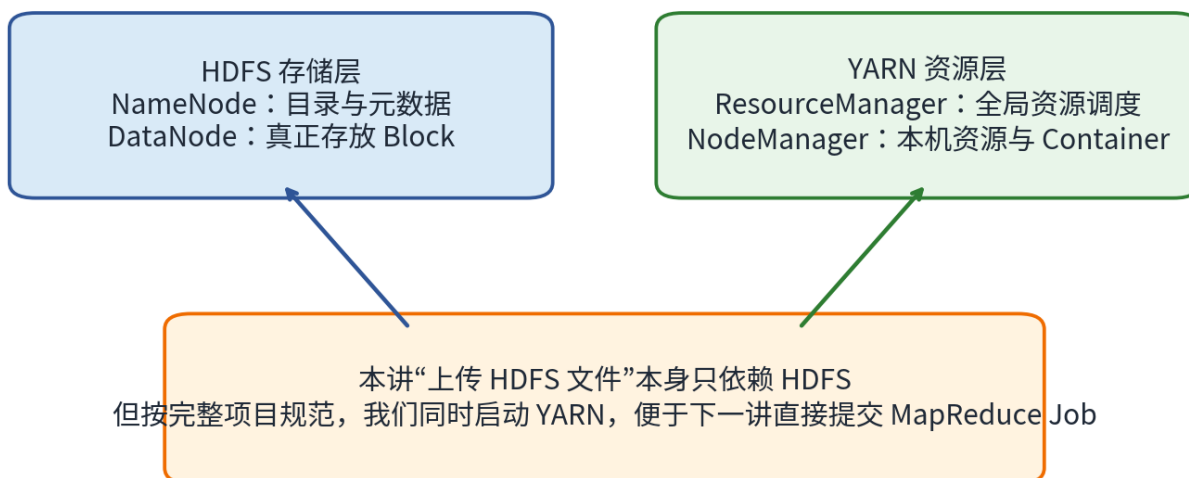
正常现象

刚开机时 `jps` 可能只看到 `Jps` 自己。这不代表 Hadoop 坏了，只说明 HDFS/YARN 守护进程还没启动。

如果 `hadoop: command not found`

先检查 `~/.bashrc` 中的 `JAVA_HOME`、`HADOOP_HOME`、`PATH` 是否还在；执行 `source ~/.bashrc` 后重试。不要直接重装 Hadoop。

本讲会启动哪些 Hadoop 服务？



单机伪分布式环境中，这四个守护进程可能都运行在同一台 Ubuntu 虚拟机上。

5. 正式启动 HDFS 和 YARN

5.1 启动 HDFS

```
start-dfs.sh
```

在单机伪分布式环境中，常见结果是启动 NameNode、DataNode（有时还会有 SecondaryNameNode）。

5.2 启动 YARN

```
start-yarn.sh
```

YARN 会启动 ResourceManager 和 NodeManager。严格来说，今天上传文件只需要 HDFS；但我们按完整项目规范同时启动 YARN，为下一讲 MapReduce 直接做好准备。

5.3 用 jps 验证，而不是只看启动脚本有没有输出

```
jps
```

进程	它负责什么	本讲是否关键
NameNode	HDFS 目录、文件到 Block 的元数据	关键
DataNode	真正保存 HDFS Block	关键
ResourceManager	YARN 全局资源调度	下一讲更关键
NodeManager	本机 Container 与资源	下一讲更关键
Jps	Java 进程查看工具	只是检查工具

特别提醒

start-dfs.sh / start-yarn.sh 的终端输出并不是最终证据。即使脚本输出很少，甚至看起来“没反应”，也应该以 jps、dfsadmin、yarn node 的实际状态为准。

6. 健康验证：服务“启动”不等于服务“健康”

```
# HDFS 健康信息：容量、Live DataNodes 等
hdfs dfsadmin -report

# 查看是否处于 Safe Mode
hdfs dfsadmin -safemode get

# YARN 节点列表
yarn node -list
```

单机环境中，hdfs dfsadmin -report 通常应看到 1 个 Live DataNode；yarn node -list 通常应看到 1 个 RUNNING 节点。

Safe Mode 是什么

NameNode 刚启动时可能短暂处于 Safe Mode，用来等待足够 Block 报告。Safe Mode 下 HDFS 通常不允许正常写入。若它长期不退出，再排查 DataNode/Block 状态，而不是第一时间强制 leave。

7. 设计 HDFS 项目目录：现在就把规范定好

不要把所有文件都丢到 HDFS 根目录。我们从综合项目开始就使用清晰的分层目录。

```
/bigdata-capstone/  
├── raw/  
│   ├── users/  
│   ├── products/  
│   └── orders/  
├── output/  
│   ├── mapreduce/  
│   └── spark/  
└── tmp/
```

raw 保存“原始输入”；output 保存不同计算引擎的结果；tmp 保存临时数据。第 2 讲只创建目录和上传 raw。

```
# 创建 HDFS 目录  
hdfs dfs -mkdir -p /bigdata-capstone/raw/users  
hdfs dfs -mkdir -p /bigdata-capstone/raw/products  
hdfs dfs -mkdir -p /bigdata-capstone/raw/orders  
hdfs dfs -mkdir -p /bigdata-capstone/output/mapreduce  
hdfs dfs -mkdir -p /bigdata-capstone/output/spark  
hdfs dfs -mkdir -p /bigdata-capstone/tmp  
  
# 查看结构  
hdfs dfs -ls /bigdata-capstone  
hdfs dfs -ls -R /bigdata-capstone
```

为什么不直接 /input?

真实项目里数据类型会越来越多。把 users/products/orders 分开，后面 Join、权限、保留周期、数据质量检查都会更清楚。

8. 上传三张历史表到 HDFS

先确认本地文件，再上传。注意左边是 Linux 本地路径 data/...，右边是 HDFS 路径 /bigdata-capstone/...。

```
# 本地先确认
ls -lh data/users.csv data/products.csv data/orders_history.csv

# 上传到 HDFS
hdfs dfs -put -f data/users.csv /bigdata-capstone/raw/users/
hdfs dfs -put -f data/products.csv /bigdata-capstone/raw/products/
hdfs dfs -put -f data/orders_history.csv /bigdata-capstone/raw/orders/
```

-f 是什么

这里的 -f 表示目标存在同名文件时覆盖。学习阶段为了重复实验方便使用；生产环境不要无脑覆盖重要原始数据。

数据从 Starter Pack 到 HDFS 的完整路径



每一步都要有“可观察证据”，不要只相信命令没有报错。

8.1 第一轮验证：HDFS 中真的出现文件了吗？

```
hdfs dfs -ls -R /bigdata-capstone/raw
```

你应该看到 3 个 CSV 文件。此时只能证明“目录项存在”，还不能证明内容正确。

9. 第二轮验证：文件内容、行数和大小是否一致

9.1 看内容

```
# users 表
hdfs dfs -cat /bigdata-capstone/raw/users/users.csv

# products 表
hdfs dfs -cat /bigdata-capstone/raw/products/products.csv

# orders 只看前几行，避免以后大文件刷屏
hdfs dfs -cat /bigdata-capstone/raw/orders/orders_history.csv | head
```

9.2 对比行数

```
# 本地行数
wc -l data/users.csv data/products.csv data/orders_history.csv

# HDFS 行数
hdfs dfs -cat /bigdata-capstone/raw/users/users.csv | wc -l
hdfs dfs -cat /bigdata-capstone/raw/products/products.csv | wc -l
hdfs dfs -cat /bigdata-capstone/raw/orders/orders_history.csv | wc -l
```

Starter Pack 中 users.csv 应为 6 行（1 行表头 + 5 行用户），products.csv 应为 6 行，orders_history.csv 应为 13 行。

9.3 看 HDFS 文件大小

```
hdfs dfs -du -h /bigdata-capstone/raw
hdfs dfs -du -h /bigdata-capstone/raw/orders
```

为什么三张表都很小？

这是教学数据。真实企业可能是 GB/TB/PB 级。我们先用可手算的小数据验证正确性，再把同样的数据流迁移到大规模系统。

10. 更严格的验证：本地文件和 HDFS 内容完全一致吗？

行数相同仍可能内容不同。我们可以用哈希值做一次“内容指纹”对比。

```
# 本地文件 MD5
md5sum data/orders_history.csv

# 把 HDFS 内容通过管道交给 md5sum
hdfs dfs -cat /bigdata-capstone/raw/orders/orders_history.csv | md5sum
```

怎么看结果

两行最左侧的 MD5 值一致，说明字节内容一致。这个方法非常适合检查“上传是否被篡改”。

如果你的文本文件在 Windows 与 Linux 之间发生过 CRLF/LF 换行转换，哈希可能不同；这时再用 cat、wc、文件内容逐项确认。

10.1 下载回来验证（可选但推荐做一次）

```
mkdir -p verify_download
hdfs dfs -get -f /bigdata-capstone/raw/orders/orders_history.csv verify_download/
ls -lh verify_download/
head verify_download/orders_history.csv
```

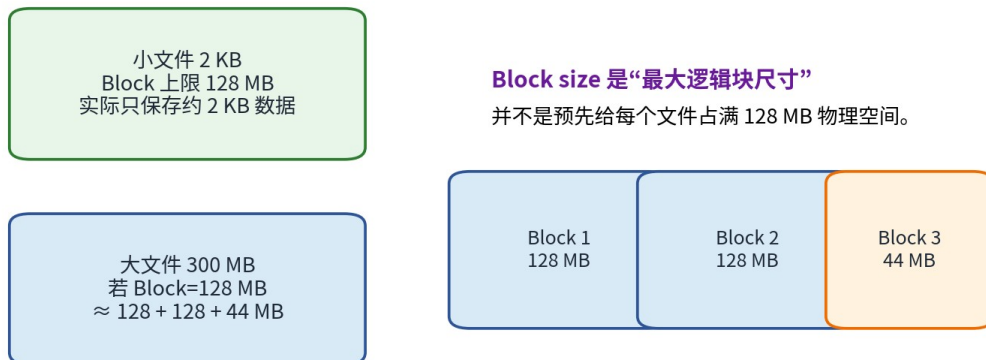
这一步让你同时练会 HDFS 的反向操作：put 是本地 → HDFS，get 是 HDFS → 本地。

11. 观察 HDFS Block：把第一个“底层原理”真正看见

```
# 查看配置的逻辑 Block 大小（通常显示字节数）
hdfs getconf -confKey dfs.blocksize
```

```
# 查看文件、Block、Block 位置
hdfs fsck /bigdata-capstone/raw -files -blocks -locations
```

HDFS Block：逻辑切分，不是“每个小文件都硬占 128 MB”



观察命令：hdfs getconf -confKey dfs.blocksize + hdfs fsck ... -files -blocks -locations

如果 dfs.blocksize 是 134217728 字节，也就是 128 MB。我们的 CSV 远小于 128 MB，所以每个文件通常只需要 1 个逻辑 Block。

关键误区

一个 2 KB 小文件“占用一个 Block”不等于它物理上硬占满 128 MB。Block size 是切分上限；实际只保存文件真实数据和元数据。

12. 副本机制：为什么单机实验和真实集群不完全一样

```
# 查看默认副本因子
hdfs getconf -confKey dfs.replication
```

```
# 查看 HDFS 文件列表（文件行中会显示 replication）
hdfs dfs -ls /bigdata-capstone/raw/orders
```

```
# fsck 还能看到 Block 位置
hdfs fsck /bigdata-capstone/raw/orders -files -blocks -locations
```

真实多节点集群里，一个 Block 可以放到多个 DataNode 上形成副本，提高容错能力。

单机伪分布式的特殊情况

如果你的 VM 只有 1 个 DataNode，最常见做法是把 dfs.replication 配成 1。若配置成 3，HDFS 也不可能凭空在同一 DataNode 上制造 3 个“独立节点副本”，因此可能出现 under-replicated 提示。

13. 可选观察：NameNode / YARN Web UI

命令行已经足够完成本讲，但 Web UI 能帮助你形成图形化心智模型。若虚拟机网络允许从 Windows 访问，可尝试：

```
NameNode UI:  http://<虚拟机 IP>:9870
YARN UI:      http://<虚拟机 IP>:8088
```

在 NameNode UI 中重点看 Live Nodes、容量、文件系统浏览入口；在 YARN UI 中此时可能还没有 Application，因为我们还没提交 MapReduce Job。

访问不了不要卡住

Web UI 受虚拟机网络模式、防火墙和端口映射影响。命令行验证正常时，本讲可以继续，不要因为浏览器打不开 UI 就反复重装 Hadoop。

本讲验收：不是“命令跑完”，而是“四层证据齐全”

1. 本地证据：data/*.csv 存在，行数正确
2. 服务证据：jps / dfsadmin / yarn node 正常
3. HDFS 证据：-ls / -cat / -du / fsck 正常
4. 业务证据：文件内容与 Starter Pack 一致

以后 MapReduce、Spark、Flink 都沿用同样的“输入 → 执行 → 结果 → 业务”验收思想。

14. 常见错误：按“层”排查，不要乱试命令

现象	最可能层次	先做什么
hdfs: command not found	环境变量	which hdfs; source ~/.bashrc
Call From ... connection refused	HDFS 服务	jps; 看 NameNode/DataNode
SafeModeException	HDFS 状态	hdfs dfsadmin -safemode get
put: No such file or directory	本地路径	pwd; ls data
HDFS 里找不到文件	HDFS 路径	hdfs dfs -ls -R /bigdata-capstone
Permission denied	HDFS 权限	hdfs dfs -ls -d /bigdata-capstone; 确认用户

文件已存在	重复上传	学习阶段用 -put -f 或先删除目标文件
fsck 显示 under-replicated	副本/节点数	检查 dfs.replication 与 Live DataNodes

排错顺序

先问“这是 Windows、本地 Linux、HDFS、YARN 还是业务数据的问题？”先定位层次，再处理。不要看到任何错误都先重启虚拟机。

15. 本讲最终验收：请逐项打勾

- ☐ VS Code 左下角确认已连接 Ubuntu Remote SSH。
- ☐ ~/bigdata-capstone/data 下三张 CSV 存在。
- ☐ jps 能看到 NameNode、DataNode、ResourceManager、NodeManager（按你的单机配置为准）。
- ☐ hdfs dfsadmin -report 有 Live DataNode。
- ☐ yarn node -list 有 RUNNING NodeManager。
- ☐ /bigdata-capstone/raw 中三张表已上传。
- ☐ HDFS cat / wc -l 结果和本地数据一致。
- ☐ hdfs fsck 能看到文件 Block 与 locations。
- ☐ 能解释“本地路径”和“HDFS 路径”为什么不是一回事。
- ☐ 能解释 Block size、实际文件大小、副本三者的区别。

15.1 结束本次实验：停止服务

```
# 先停 YARN
stop-yarn.sh

# 再停 HDFS
stop-dfs.sh

# 验证
jps
```

停止服务不会删除你已经上传到 HDFS 的数据。第二天重新启动虚拟机后，再执行 start-dfs.sh 和 start-yarn.sh，HDFS 文件仍应存在。

为什么要养成“启停闭环”

教学 VM 资源有限。每次明确知道“服务有没有启动、什么时候停止”，能显著减少后续端口占用、残留进程和误判问题。

16. 练习：不要只照抄命令

- 为什么在 ~/bigdata-capstone/data 能看到 orders_history.csv，但执行 hdfs dfs -ls ~/bigdata-capstone/data 不一定有意义？
- 如果 dfs.blocksize=128 MB，一个 2 KB 的 users.csv 为什么不会真的占满 128 MB 磁盘？

3. 为什么今天上传 HDFS 文件严格来说不需要 YARN，但我们仍启动了 YARN？
4. 如果 jps 有 NameNode，但没有 DataNode，hdfs dfs -put 可能出现什么问题？你第一步查什么？
5. 如果单机只有 1 个 DataNode，而 dfs.replication=3，为什么可能出现 under-replicated？
6. 请写出检查 “orders_history.csv 本地和 HDFS 内容是否一致” 的两种方法。

参考答案

1. 因为 ~/... 是 Linux 本地路径；HDFS 使用自己的命名空间，应该用 /bigdata-capstone/... 之类的 HDFS 路径。
2. 128 MB 是逻辑 Block 的最大切分尺寸，不是对每个小文件预分配 128 MB 物理空间。
3. YARN 管计算资源，不负责 HDFS 文件写入；但下一讲要提交 MapReduce Job，提前按完整项目规范启动有助于形成稳定流程。
4. 可能无法正常写入数据块，或出现无可用的 DataNode 等问题。先用 jps 和 hdfs dfsadmin -report 确认 DataNode 状态。
5. 副本要分布到不同 DataNode；只有 1 个 DataNode 无法满足 3 副本目标，因此会欠副本。
6. 方法一：本地和 HDFS 分别 wc -l / cat 对比；方法二：md5sum 本地文件，并对 hdfs dfs -cat 的输出计算 md5sum。

17. 本讲总结：你第一次把“数据”真正放进了大数据系统

到这里，综合项目的数据不再只是本地 CSV，而是已经进入 HDFS，成为后续分布式计算的统一输入。

```
本地 Starter Pack
    ↓ hdfs dfs -put
HDFS /bigdata-capstone/raw
    ↓
下一讲：MapReduce 读取 orders_history.csv
    ↓
按城市统计 PAID 订单销售额
    ↓
与第 1 讲手算标准答案比较
```

下一讲预告

第 3 讲会用 Java MapReduce 完成第一个真实离线任务：按城市统计 PAID 订单销售额。你会亲眼看到 InputSplit → Mapper → Shuffle → Reducer → HDFS Output 如何与这个综合项目的数据对应起来。